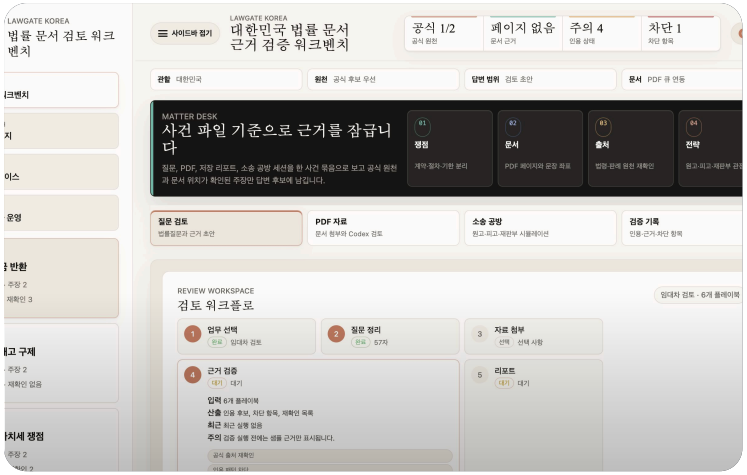


AI PRODUCT / DOCUMENT RAG 01

LAW GATE

법률 문제를 쟁점 분해, 멀티에이전트 검토, 공식 근거 재검증까지 연결한 워크벤치 제품.

첫 질문

“더 그럴듯한 답보다, 근거 없는 답을 먼저 막으려면 어떻게 해야 할까?”

법률 문서와 증거 자료는 양이 많고 표현도 어렵습니다. 필요한 근거를 찾지 못한 채 답을 만들면 위험합니다. 그래서 답변보다 근거를 먼저 확인하는 워크벤치를 만들었습니다.

01
문제
PDF와 HWP에서 쟁점을 찾고, 근거 없는 주장과 과한 법률 조언을 막으면서도 누구나 결과를 이해해야 했습니다.

02
작게 시작하기
처음에는 HTML·CSS·JS와 Node로 핵심 흐름만 만들었습니다. 필요한 기능이 확인된 뒤 Codex CLI, RAG, Neo4j, 법령 API, PDF 처리를 붙였습니다.

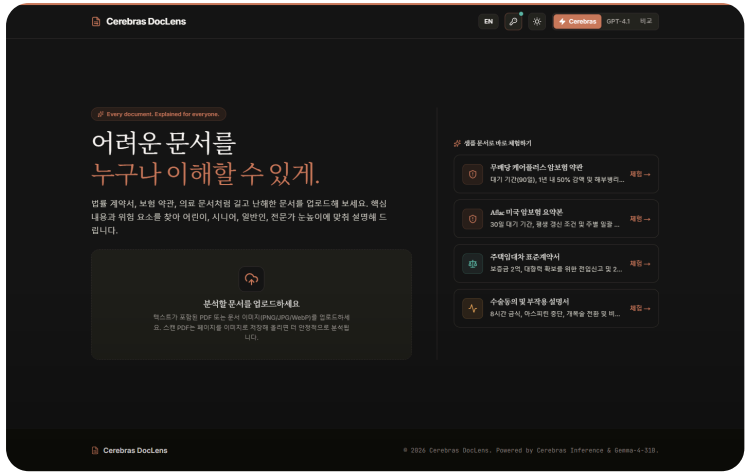
03
문제에서 배운 점
PDF 33개, 약 921MB를 처리하면서 느린 속도·Windows 오류·Codex 제한·캐시 부재가 드러났습니다. 문제를 하나씩 질문으로 바꾸고 원인을 확인했습니다.

04
진행 순서
질문을 쪼갠 뒤 PDF·HWP를 받고, 로컬 RAG로 근거를 찾았습니다. 공식 법령(법제처 API)을 다시 확인하고 원고·피고·판사 Agent가 같은 자료를 보며 검토하게 했습니다.

05
Agent 작업 기준
Agent마다 역할과 출력 형식을 정했습니다. 법률 기준, JSON Schema, 증거 원장, 입증되지 않은 부분을 나눠 결과가 섞이지 않게 했습니다.

06
제품으로 연결하기
파일 hash 캐시, 배치 처리, 중간 저장, 빠른 모드·정밀 모드, 4단계 검토, 세션 저장, PDF 내보내기를 연결했습니다.

07
결과
PDF·HWP 문서에서 쟁점을 찾고, 공식 근거를 다시 확인하며, 일반 사용자도 결과를 이해할 수 있는 법률 문서 검토 제품으로 완성했습니다.



AI PRODUCT / DOCUMENT INTELLIGENCE 02

CEREBRAS DOCLENS

복잡한 PDF를 여러 AI 에이전트가 분석해 핵심·위험·비용·일정을 쉽게 설명하는 문서 이해 제품.

첫 질문

“복잡한 문서를 쉽게 읽고 판단하게 하면서도, 분석에 오래 걸리지 않게 하려면 어떻게 할까?”

긴 문서의 위험·금액·기한을 쉽게 풀어 보여주는 것만큼, 사용자가 결과를 오래 기다리지 않게 하는 것도 중요했습니다. 그래서 어려운 내용을 핵심·위험·비용·일정으로 나누고, Cerebras API로 여러 Agent를 동시에 실행하는 구조를 먼저 검증했습니다.

01
문제
복잡하고 긴 문서를 누구나 읽고 이해하며, 필요한 다음 행동까지 판단할 수 있게 하고 싶었습니다.

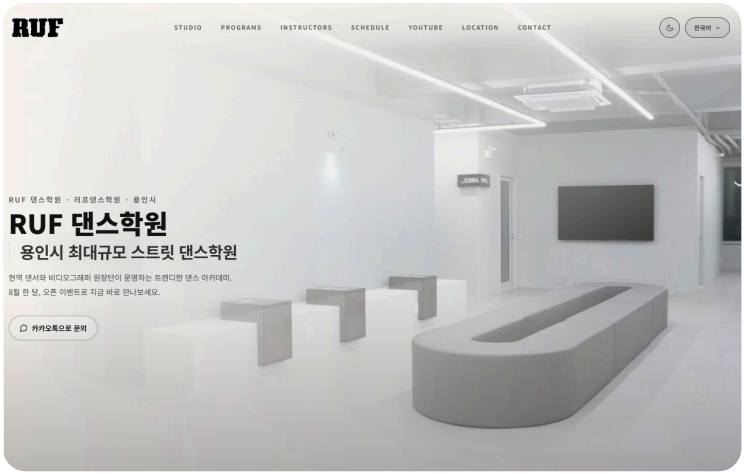
02
진행 순서
모든 Agent가 같은 문서를 반복해서 읽으면 토큰과 시간이 낭비됩니다. Intake Agent가 먼저 문서를 읽고, Summary·Risk·Cost·Timeline Agent가 같은 근거를 나눠 동시에 분석하도록 구성했습니다.

03
사용한 도구
Next.js, Cerebras API, Gemma 4 31B, PDF/Image parsing, JSON schema, Vercel을 연결했습니다. 여러 Agent를 동시에 실행해 기다리는 시간을 줄이는 것이 도구 선택의 기준이었습니다.

04
문제 해결
Vercel에서 PDF를 읽지 못했을 때 모델 문제가 아니라 실행 환경 문제로 나눠서 확인했습니다. 필요한 설정을 보완하고, OCR은 이미지 업로드부터 안정적으로 동작하게 범위를 조정했습니다.

05
사람의 판단
어디까지 쉽게 풀어쓸지, 어떤 원문을 함께 보여줄지, 사용자가 무엇을 판단해야 할지를 정했습니다. 결과는 직접 실행해 보고 다시 고쳤습니다.

06
결과
Cerebras API로 여러 Agent를 동시에 실행하고, 핵심·위험·비용·일정과 원문 근거를 함께 보여주는 문서 분석 제품으로 완성했습니다.



PRODUCT / WEB

03

RUF DANCE ACADEMY WEB

브랜드의 에너지와 움직임을 웹 경험으로 번역한 사이트.

핵심 질문

“운영자가 개발자에게 매번 부탁하지 않아도, 계속 쓸 수 있는 웹을 어떻게 만들까?”

RUF는 한 번 보고 끝나는 홍보 페이지가 아니었습니다. 시간표와 강사, 주간 수업, 상담·위치 정보가 계속 바뀌었습니다. 운영자가 직접 고치고 방문자가 바로 상담까지 갈 수 있는 구조를 먼저 생각했습니다.

01

문제

댄스 아카데미의 브랜드 소개, 수업 찾기, 위치 확인, 상담 연결을 하나의 웹 경험으로 묶어야 했습니다.

02

제품 만들기

Next.js·React·TypeScript로 브랜드 화면, 다국어, 일정, 강사, 상담, 위치, FAQ, SEO, 개인정보 페이지를 처음부터 만들었습니다.

03

Google Sheet를 고른 이유

별도 관리자 페이지 대신 운영자가 익숙한 Google Sheets를 CMS로 쓰기로 했습니다. 코드 수정 없이 시간표·주간 수업·영상·강사 정보를 바꿀 수 있고, 지금 규모에는 이 방식이 더 작고 빠르게 운영할 수 있었습니다.

04

화면에서 확인한 문제

브라우저에서 첫 메시지가 밀리고, 한글 줄바꿈이 어색하고, 모바일에서 가로 스크롤이 생기는 문제를 확인했습니다. 강사 캐러셀처럼 보기에는 화려하지만 탐색을 방해하는 요소도 단순하게 바꿨습니다.

05

멈추지 않게 하기

Sheets가 비거나 네트워크가 끊겨도 화면이 비지 않도록 정적 fallback을 준비했습니다. 주소 검색이 실패할 때는 저장된 좌표를 사용해 지도도 계속 보이게 했습니다.

06

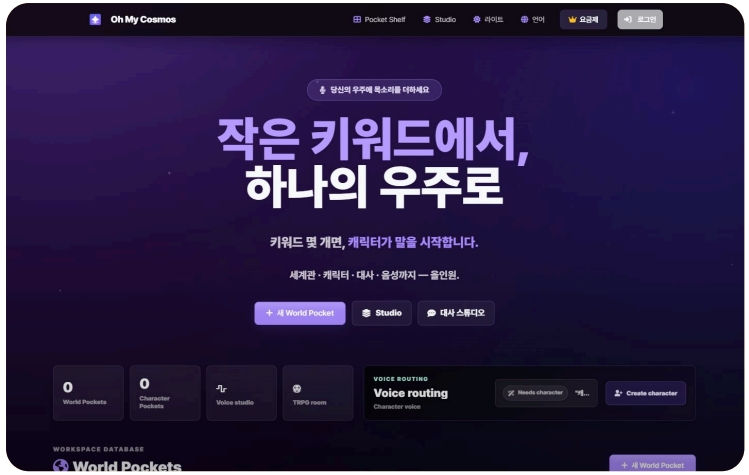
운영에 맞추기

실제 일정이 바뀌면 만료된 신청 버튼과 이벤트 문구를 남겨두지 않았습니다. 시설 사진, 이미지 로딩, 모바일 CTA도 운영 상황에 맞춰 다시 확인했습니다.

07

결과

기획과 UI부터 다국어·SEO·지도·상담·운영 데이터·반응형 QA·배포까지 연결했습니다. 5월에 시작해 7월 말까지 78개 커밋으로 실제 운영 가능한 웹 제품을 만들었습니다.



PRODUCT / KNOWLEDGE

OH MY COSMOS

인디 게임 팀의 흩어진 기획을 캐릭터·대사·AI 보이스로 연결하는 제작 공간.

핵심 질문

"흩어진 세계관과 캐릭터 아이디어를 AI가 실제 게임 제작 자산으로 연결하려면 어떻게 해야 할까?"

인디 게임 팀에서 Discord·Notion·회의에 나온 설정과 대사가 흩어졌습니다. 캐릭터 설정이 정리되지 않으면 기획과 게임 사운드도 함께 멈췄습니다. 그래서 이 문제를 협업 캔버스와 AI 제작 흐름으로 만들고, 제작하는 게임에 직접 적용해 보기로 했습니다.

01

문제

작은 팀은 기획자·작가·사운드 인력이 부족합니다. 세계관과 캐릭터, 대사, 음성을 관리하기 어렵고 자료를 옮기는 동안 설정도 쉽게 어긋났습니다.

02

하나로 이어진 흐름

Figma·Notion·ChatGPT·TTS를 따로 쓰면 회의 내용과 제작 결과 사이가 끊깁니다. 그래서 협업 캔버스에서 시작해 AI가 정리하고, Persona·Dialogue·TTS·JSON/ZIP Export까지 이어지게 했습니다.

03

Agent가 하는 일

AI가 캔버스·회의 기록·문서를 읽고 세계관·캐릭터·관계·대사 후보로 정리합니다. 원문은 남겨두고 사람이 확인한 결과만 제작 자산으로 넘깁니다.

04

먼저 실제 게임에 적용

처음부터 모든 창작 분야를 해결하려 하지 않았습니다. 제작 중인 게임의 다섯 캐릭터에 역할·성격·대사·보이스 톤·관계를 적용하며 먼저 검증했습니다.

05

작은 팀부터 시작

기본 캔버스는 가볍게 열고, AI 정리·TTS·엔진 Export는 팀 기능으로 확장할 수 있게 했습니다. 이후 인디 게임뿐 아니라 웹소설·웹툰·애니메이션 팀도 사용할 수 있도록 범위를 열어두었습니다.

06

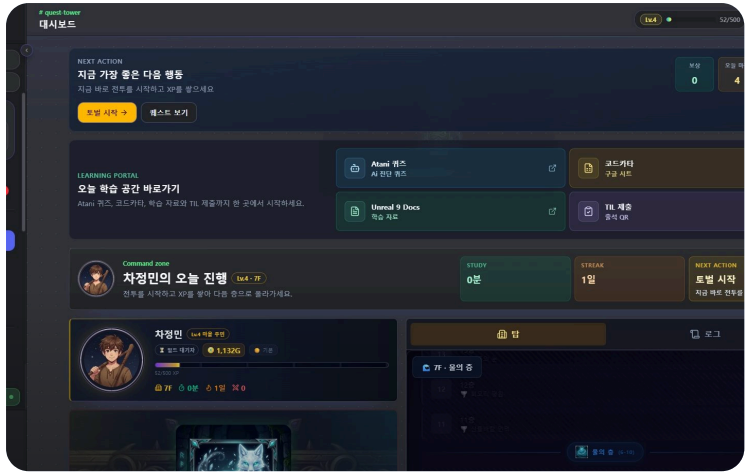
제품 흐름

Cloudflare 기반 서비스에 캐릭터·세계관·대사·TTS·엔진 Export를 연결했습니다. 음성은 라이선스를 확인할 수 있는 API를 사용해 저작권 위험도 함께 살폈습니다.

07

결과

회의에서 나온 아이디어를 실제 제작 자산으로 이어주는 AI 협업 공간으로 정의했습니다. 제작 중인 게임에 적용하며 제작 속도와 세계관의 일관성을 확인하고 있습니다.



PRODUCT / LEARNING

05

QUEST TOWER

학습·퀘스트·성장 루프를 게임형 인터페이스로 연결한 제품.

핵심 질문

“학습 관리 화면을 넘어, 학생이 계속 다음 행동을 하게 만드는 제품을 어떻게 만들까?”

Unreal 부트캠프에서 학생이 학습을 계속 이어가게 만들고 싶었습니다. 기능을 많이 넣기보다 다음 행동과 보상이 분명해야 한다고 봤습니다.

01

문제

학생과 튜터의 학습 운영을 한곳에 모으고, 퀘스트 완료 → XP 획득 → 탑 성장으로 이어지는 반복 흐름을 만들어야 했습니다.

02

제품 만들기

요구사항과 인터뷰로 범위를 정한 후 화면·API·DB를 한 기능씩 연결했습니다. 로그인, 퀘스트, 타이머, 랭킹, C++ 채점, 보상 시스템까지 붙였습니다.

03

다음 행동을 쉽게 만들기

학생이 화면을 보고 바로 무엇을 해야 할지 알 수 있어야 했습니다. 그래서 받을 보상이 있으면 ‘보상 받기’를 먼저 보여주고, 없으면 ‘학습 시작’을 바로 보여줬습니다.

04

버그 신고 → 바로 개선

사용자가 불편한 점을 바로 신고할 수 있는 기능을 만들었습니다. 신고가 접수되면 내부 포인트를 보상하고, 내용을 확인해 다음 개선 작업으로 바로 이어지게 했습니다.

05

꼬이지 않게 하기

여러 요청이 동시에 들어와도 XP와 골드가 덮어써지지 않게 서버에서 한 번에 처리하고 기록했습니다. 실패하면 이전 상태로 되돌렸습니다.

06

운영하며 개선하기

운영 중에는 유저들의 실시간 반응을 살펴 불편한 점을 바로 고쳤습니다. 중간중간 이벤트를 열어 다시 찾아올 이유도 만들었습니다.

07

결과

기획부터 DB·UI·외부 API·배포·운영까지 연결했습니다. 질문하고 고치고 확인하는 과정을 반복해 학습 제품을 완성했습니다.



OPEN SOURCE / UE5

06

UNREAL AGENT

Unreal Editor를 읽고, 조작하고, 검증하는 AI 에이전트.

핵심 질문

"AI Agent가 Unreal Editor를 직접 읽고 움직이게 하려면 어떤 구조가 필요할까?"

AI로 Unreal 코드를 작성하는 데서 끝내고 싶지 않았습니다. Agent가 Editor의 상태를 읽고, 도구를 고르고, 바뀐 결과를 다시 확인하는 흐름을 만들고 싶었습니다.

01

문제

개발자가 AI를 쓰는 것이 아니라, AI Agent가 Unreal Editor 안에서 직접 작업할 수 있어야 했습니다.

02

연결 구조

MCP HTTP·stdio를 Codex CLI·Claude CLI와 연결했습니다. OpenAI·Anthropic 호환 Provider도 바꿔 쓸 수 있게 했습니다.

03

Unreal 도구

Blueprint·UMG·Material·Niagara·Control Rig·Asset·Viewport 작업을 Agent가 부를 수 있게 만들고, RAG와 Vision도 연결했습니다.

04

작업 흐름

Agent가 현재 상태를 확인하고 작업을 호출합니다. 작업이 끝나면 결과를 다시 읽어 다음 행동을 정하게 했습니다.

05

안전 장치

변경 전에 dryRun으로 결과와 범위를 먼저 확인하게 했습니다. 실수한 변경이 바로 실행되지 않게 한 것입니다.

06

문제 해결

MCP discovery가 안 될 때 tools/list를 직접 확인해 연결 문제를 좁혔습니다. 원인을 확인하는 도구도 Agent 흐름의 일부로 만들었습니다.

07

결과

AI로 게임을 만든 것을 넘어, AI가 Unreal Editor를 읽고 움직이고 확인하는 도구 계층을 직접 만들었습니다.



OPEN SOURCE / UNITY

07

UNITY MCP

자연어 명령으로 Unity 씬과 에디터 작업을 움직이는 AI 개발 도구.

핵심 질문

“자연어로 Unity Editor를 조작하고, Agent의 변경을 안전하게 검증하려면 어떻게 해야 할까?”

Unity 작업을 AI에게 맡기는 것만으로는 부족했습니다. Agent가 Scene과 Asset을 읽고, 필요한 도구를 부르고, 바뀐 상태를 확인해야 실제 작업이 된다고 봤습니다.

01

문제

자연어로 명령한 Scene·Asset·Material·Lighting·Audio·Character·Animation 작업이 Unity Editor의 실제 결과로 이어져야 했습니다.

02

Unity MCP

Unity Editor의 상태를 파악하고 AI가 작업할 수 있도록 MCP 도구로 만들었습니다. Agent가 씬을 읽고 필요한 명령을 고를 수 있게 한 것입니다.

03

도구 나누기

Scene·Asset·Material·Lighting·Audio·Character·Animation을 각각 호출할 수 있는 도구로 나눠 작업 범위를 분명하게 했습니다.

04

안전 장치

변경 도구에 dryRun을 넣었습니다. 실행 전에 결과와 범위를 확인하고, 문제가 있으면 멈출 수 있게 했습니다.

05

문제 해결

오디오 authoring과 beta packaging을 추가하면서 안전 정책도 함께 정리했습니다. 기능 수보다 실제로 배포할 수 있는 흐름을 우선했습니다.

06

결과

Unity Editor를 자연어와 MCP로 움직이고, 실행 전 확인과 실행 후 검토까지 연결하는 도구를 만들었습니다.